```
import shap
import joblib
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
model = joblib.load("xgb_rul_model.pkl")
```

```
train_df = pd.read_csv("train_engineered.csv")
test_df = pd.read_csv("test_engineered.csv")
```

```
X_train = train_df.drop(columns=['unit_id', 'RUL'])
X_test = test_df.drop(columns=['unit_id', 'RUL'])

y_train = train_df['RUL']
y_test = test_df['RUL']
```

```
from xgboost import XGBRegressor

model = XGBRegressor(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

model.fit(X_train, y_train)
```

```
XGBRegressor

XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=0.8, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.05, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=300,
             n_jobs=None, num_parallel_tree=None, ...)
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

preds = model.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, preds))
mae = mean_absolute_error(y_test, preds)

print("RMSE:", rmse)
print("MAE:", mae)
```

```
RMSE: 44.09053148811425
MAE: 33.304318387600695
```

```
importance = model.feature_importances_

feature_importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': importance
}).sort_values(by='Importance', ascending=False)

print(feature_importance.head(10))
```

```
      Feature  Importance
19  s11_roll5    0.473606
20  s12_roll5    0.256153
22   s4_roll5    0.139051
23   s7_roll5    0.037531
```

```
0       cycle    0.012964
14        s14    0.009617
26    s6_lag1    0.007480
10         s9    0.007067
7          s6    0.005451
15        s15    0.004692
```

```
engineered_cols = [col for col in X_train.columns if "roll" in col or "lag" in col or "min" in col or "max" in col]

print(engineered_cols)
print("Count:", len(engineered_cols))
```

```
['s11_roll5', 's12_roll5', 's6_roll5', 's4_roll5', 's7_roll5', 's11_lag1', 's12_lag1', 's6_lag1', 's4_lag1', 's7_lag1']
Count: 10
```

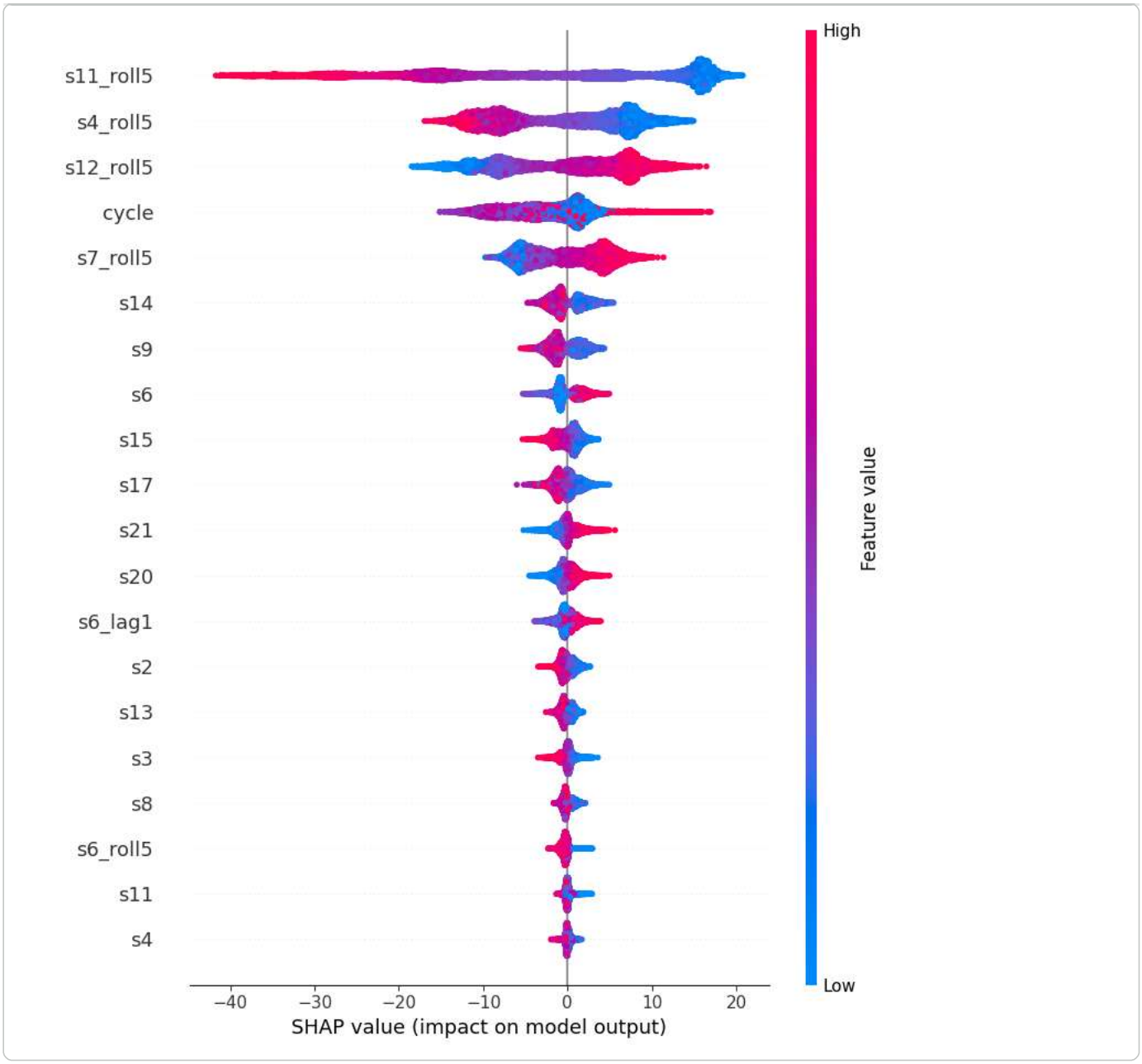
```
import joblib

joblib.dump(model, "xgb_rul_model.pkl")
joblib.dump(X_train.columns.tolist(), "feature_schema.pkl")
```

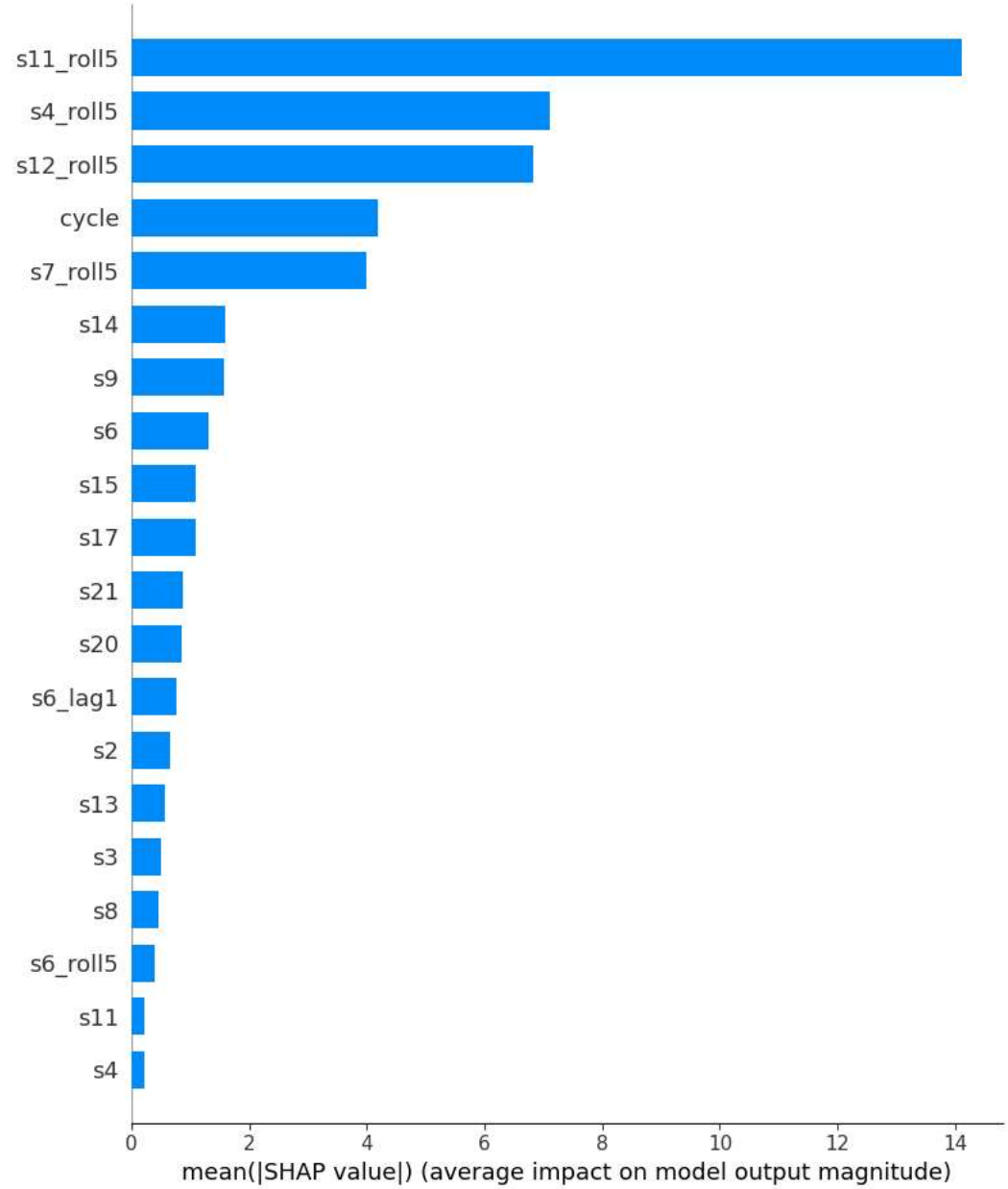
```
['feature_schema.pkl']
```

```
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
```

```
shap.summary_plot(shap_values, X_test)
```




```
shap.summary_plot(
    shap_values,
    X_test,
    plot_type="bar"
)
```




```
shap_importance = np.abs(shap_values).mean(axis=0)

feature_importance = pd.DataFrame({
    "Feature": X_test.columns,
    "SHAP Importance": shap_importance
})

feature_importance = feature_importance.sort_values(
    by="SHAP Importance",
    ascending=False
)

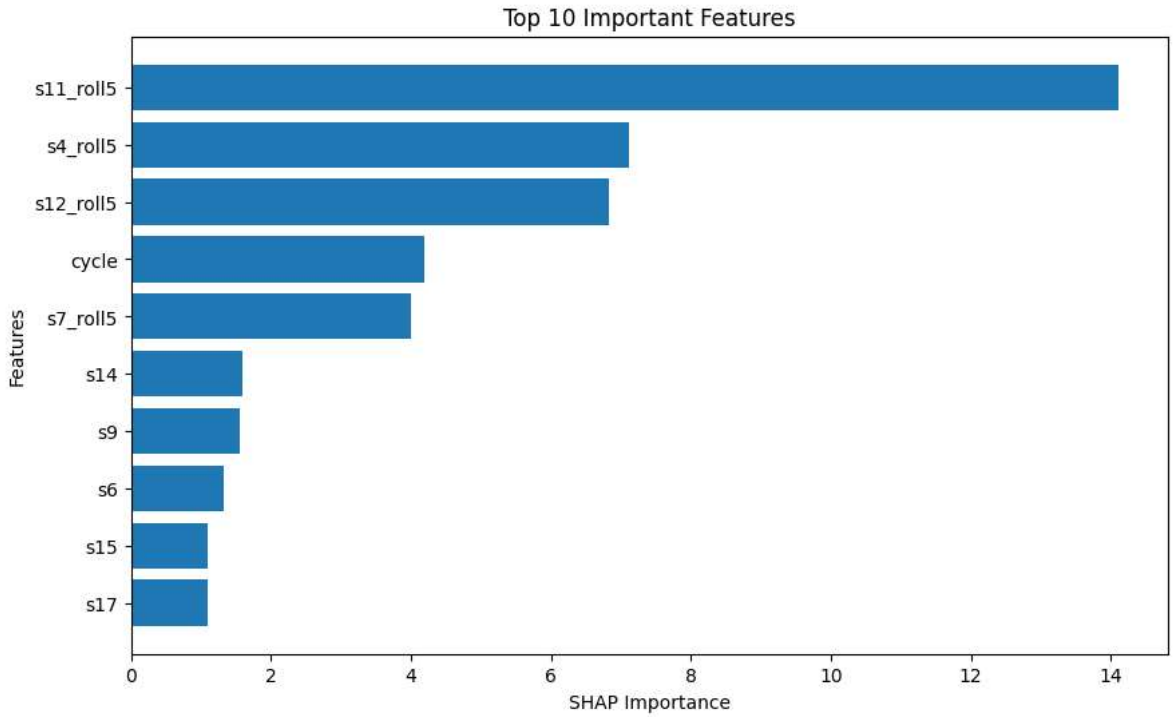
top10 = feature_importance.head(10)

print(top10)
```

```
      Feature  SHAP Importance
19  s11_roll5        14.104960
22   s4_roll5         7.114180
20  s12_roll5         6.836903
0       cycle         4.193143
23   s7_roll5         4.000366
14        s14         1.600380
10         s9         1.560526
7          s6         1.319376
```

```
15        s15         1.095005
16        s17         1.090840
```

```
plt.figure(figsize=(10,6))
plt.barh(
    top10["Feature"],
    top10["SHAP Importance"]
)
plt.xlabel("SHAP Importance")
plt.ylabel("Features")
plt.title("Top 10 Important Features")
plt.gca().invert_yaxis()
plt.show()
```




```
import numpy as np

def get_root_causes(sample_idx, top_k=3):
    """
    Extract top-k root cause features using SHAP values
    """

    sample_shap = shap_values[sample_idx]

    # Sort by highest absolute impact
    sorted_idx = np.argsort(np.abs(sample_shap))[::-1]

    root_causes = []

    for idx in sorted_idx[:top_k]:
        root_causes.append({
            "feature": X_test.columns[idx],
            "impact": float(sample_shap[idx]),
            "sensor_value": float(X_test.iloc[sample_idx, idx])
        })

    return root_causes
```

```
def classify_severity(impact):
    """
    Convert SHAP magnitude into severity level
    """

    impact = abs(impact)

    if impact >= 10:
```

```
        return "High"
    elif impact >= 5:
        return "Medium"
    else:
```

```
sensor_map = {
    "s11": "Thermal Degradation",
    "s12": "Combustion Instability",
    "s4": "Pressure Fluctuation",
    "s7": "Efficiency Loss",
    "s6": "Airflow Variation",
    "cycle": "Lifecycle Wear"
}
```

```
def map_sensor_issue(feature):
    """
    Map raw feature to readable issue
    """

    base_sensor = feature.split("_")[0]

    return sensor_map.get(base_sensor, "Unknown Issue")
```

```
def get_diagnosis(sample_idx, top_k=3):
    """
    Generate root-cause diagnosis
    """

    causes = get_root_causes(sample_idx, top_k)

    diagnosis = []

    for cause in causes:
        diagnosis.append({
            "feature": cause["feature"],
            "issue": map_sensor_issue(cause["feature"]),
            "impact": cause["impact"],
            "severity": classify_severity(cause["impact"]),
            "sensor_value": cause["sensor_value"]
        })

    return diagnosis
```

```
def engine_report(sample_idx):
    """
    Complete engine health report
    """
```